

```
1 !pip install tensorflow torch torchvision
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.11.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.26.0+cpu)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.1)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.2)
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.12.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.2.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.81.1)
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.3)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.44.0)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.19.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2025.11.11)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (3.1.0)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (3.8.1)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (0.7.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (3.1.3)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.10.0) (0.1.2)
```

```
1 import tensorflow as tf
2 import torch
3
4 print("TensorFlow:", tf.__version__)
5 print("Pytorch:", torch.__version__)
```

TensorFlow: 2.20.0
Pytorch: 2.11.0+cpu

```
1 # Exercise 1: Tensor Creation
2
3 # TensorFlow
4 import tensorflow as tf
5
6 tensor_tf = tf.constant([
7     [1, 2, 3],
8     [4, 5, 6]
9 ])
10
11 print("Tensor:")
12 print(tensor_tf)
13 print()
14 print("Shape:", tensor_tf.shape)
15 print("Data Type:", tensor_tf.dtype)
16 print("Dimensions:", tensor_tf.ndim)
```

Tensor:
tf.Tensor(
[[1 2 3]
 [4 5 6]], shape=(2, 3), dtype=int32)

Shape: (2, 3)
Data Type: <dtype: 'int32'>
Dimensions: 2

```
1 # Exercise 1: Tensor Creation
2
```

```
3 # Pytorch
4 import torch
5
6 tensor_torch = torch.tensor([
7     [1, 2, 3],
8     [4, 5, 6]
9 ])
10
11 print("Tensor:")
12 print(tensor_torch)
13 print()
14 print("Shape:", tensor_torch.shape)
15 print("Data Type:", tensor_torch.dtype)
16 print("Dimensions:", tensor_torch.ndim)
```

Tensor:
tensor([[1, 2, 3],
 [4, 5, 6]])

Shape: torch.Size([2, 3])
Data Type: torch.int64
Dimensions: 2

```
1 # Exercise 2 : Basic Tensor Operations
2
3 # TensorFlow
4 import tensorflow as tf
5
6 A = tf.constant([[1,2],[3,4]])
7 B = tf.constant([[5,6],[7,8]])
8
9 print("Addition:")
10 print(A + B)
11
12 print("\nSubtraction:")
13 print(A - B)
14
15 print("\nElement-wise Multiplication:")
16 print(A * B)
17
18 print("\nMatrix Multiplication:")
19 print(tf.matmul(A,B))
```

Addition:
tf.Tensor(
[[6 8]
 [10 12]], shape=(2, 2), dtype=int32)

Subtraction:
tf.Tensor(
[[-4 -4]
 [-4 -4]], shape=(2, 2), dtype=int32)

Element-wise Multiplication:
tf.Tensor(
[[5 12]
 [21 32]], shape=(2, 2), dtype=int32)

Matrix Multiplication:
tf.Tensor(
[[19 22]
 [43 50]], shape=(2, 2), dtype=int32)

```
1 # Exercise 2 : Basic Tensor Operations
2
3 # Pytorch
4 import torch
5
6 A = torch.tensor([[1,2],[3,4]])
7 B = torch.tensor([[5,6],[7,8]])
8
9 print("Addition:")
10 print(A + B)
11
12 print("\nSubtraction:")
13 print(A - B)
14
15 print("\nElement-wise Multiplication:")
16 print(A * B)
17
18 print("\nMatrix Multiplication:")
19 print(torch.matmul(A,B))
```

Addition:
tensor([[6, 8],
 [10, 12]])

Subtraction:
tensor([[-4, -4],
 [-4, -4]])

Element-wise Multiplication:
tensor([[5, 12],
 [21, 32]])

Matrix Multiplication:
tensor([[19, 22],
 [43, 50]])

```
1 # Exercise 3: Tensor Reshaping
2
3 # TensorFlow
4 import tensorflow as tf
5
6 tensor = tf.constant([1,2,3,4,5,6,7,8,9,10,11,12])
7
8 reshape1 = tf.reshape(tensor, (3,4))
9 reshape2 = tf.reshape(tensor, (2,6))
10 flatten = tf.reshape(reshape1, (-1,))
11
12 print("3x4:")
13 print(reshape1)
14
15 print("\n2x6:")
16 print(reshape2)
17
18 print("\nFlatten:")
19 print(flatten)
```

3x4:
tf.Tensor(
[[1 2 3 4]
 [5 6 7 8]
 [9 10 11 12]], shape=(3, 4), dtype=int32)

2x6:
tf.Tensor(
[[1 2 3 4 5 6]
 [7 8 9 10 11 12]], shape=(2, 6), dtype=int32)

Flatten:
tf.Tensor([1 2 3 4 5 6 7 8 9 10 11 12], shape=(12,), dtype=int32)

```
1 # Exercise 3: Tensor Reshaping
2
3 # Pytorch
4 import torch
5
6 tensor = torch.tensor([1,2,3,4,5,6,7,8,9,10,11,12])
7
8 reshape1 = tensor.reshape(3,4)
9 reshape2 = tensor.reshape(2,6)
10 flatten = reshape1.flatten()
11
12 print("3x4:")
13 print(reshape1)
14
15 print("\n2x6:")
16 print(reshape2)
17
18 print("\nFlatten:")
19 print(flatten)
```

3x4:
tensor([[1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 10, 11, 12]])

2x6:
tensor([[1, 2, 3, 4, 5, 6],
 [7, 8, 9, 10, 11, 12]])

Flatten:
tensor([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])

```
1 # Exercise 4: Automatic Differentiation
2
3 # TensorFlow
4 import tensorflow as tf
5
6 x = tf.Variable(5.0)
7
8 with tf.GradientTape() as tape:
9     y = x**2 + 3*x + 2
```

```
10
11 dy_dx = tape.gradient(y, x)
12
13 print("y =", y.numpy())
14 print("dy/dx =", dy_dx.numpy())
```

y = 42.0

dy/dx = 13.0

```
1 # Exercise 4: Automatic Differentiation
2
3 # Pytorch
4 import torch
5
6 x = torch.tensor(5.0, requires_grad=True)
7
8 y = x**2 + 3*x + 2
9
10 y.backward()
11
12 print("y =", y.item())
13 print("dy/dx =", x.grad.item())
```

y = 42.0

dy/dx = 13.0

```
1 # Exercise 5: Neural Network Architecture
2
3 # TensorFlow
4 import tensorflow as tf
5
6 model = tf.keras.Sequential([
7     tf.keras.Input(shape=(784,)),
8     tf.keras.layers.Dense(128, activation='relu'),
9     tf.keras.layers.Dense(10, activation='softmax')
10 ])
11
12 model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

```
1 # Exercise 5: Neural Network Architecture
2
3 # Pytorch
4 import torch
5 import torch.nn as nn
6
7 class NeuralNet(nn.Module):
8     def __init__(self):
9         super().__init__()
10
11         self.fc1 = nn.Linear(784,128)
12         self.relu = nn.ReLU()
13         self.fc2 = nn.Linear(128,10)
14
15     def forward(self, x):
16         x = self.c1(x)
17         x = self.relu(x)
18         x = self.fc2(x)
19         return x
20
21 model = NeuralNet()
22
23 print(model)
24
25 total_params = sum(
26     p.numel()
27     for p in model.parameters()
28     if p.requires_grad
29 )
30
31 print("Trainable Parameters:", total_params)
```

NeuralNet(
 (fc1): Linear(in_features=784, out_features=128, bias=True)
 (relu): ReLU()

```
(fc2): Linear(in_features=128, out_features=10, bias=True)
)
Trainable Parameters: 101770
```

```
1 # Exercise 6: MNIST Digit Classification using Tensorflow
2 # Step 1: Download the Data
3 import tensorflow as tf
4
5 (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
6
7 print("Training Images:", x_train.shape)
8 print("Training Labels:", y_train.shape)
9 print("Test Images:", x_test.shape)
10 print("Test Labels:", y_test.shape)
```

Training Images: (60000, 28, 28)
Training Labels: (60000,)
Test Images: (10000, 28, 28)
Test Labels: (10000,)

```
1 # Exercise 6 Tensorflow - Step 2: Normalize the Images
2
3 x_train = x_train / 255.0
4 x_test = x_test / 255.0
5
6 print("Min value:", x_train.min())
7 print("Max value:", x_train.max())
```

Min value: 0.0
Max value: 1.0

```
1 # Exercise 6 Tensorflow - Step 3: Build Neural Network
2
3 import tensorflow as tf
4
5 model = tf.keras.Sequential([
6     tf.keras.layers.Flatten(input_shape=(28,28)),
7     tf.keras.layers.Dense(128, activation='relu'),
8     tf.keras.layers.Dense(10, activation='softmax')
9 ])
10
11 model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape` argument to the Flatten layer. It is inferred from the input data.
super().__init__(**kwargs)
Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_4 (Dense)	(None, 128)	100,480
dense_5 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)

```
1 # Exercise 6 Tensorflow - Step 4: Compile and Train
2
3 model.compile(
4     optimizer='adam',
5     loss='sparse_categorical_crossentropy',
6     metrics=['accuracy']
7 )
8
9 history = model.fit(
10     x_train,
11     y_train,
12     epochs=5,
13     validation_split=0.1
14 )
```

Epoch 1/5
1688/1688 ————— 12s 6ms/step - accuracy: 0.9235 - loss: 0.2714 - val_accuracy: 0.9600 - val_loss: 0.1346
Epoch 2/5
1688/1688 ————— 10s 6ms/step - accuracy: 0.9642 - loss: 0.1201 - val_accuracy: 0.9667 - val_loss: 0.1152
Epoch 3/5
1688/1688 ————— 14s 8ms/step - accuracy: 0.9751- loss: 0.0822 - val_accuracy: 0.9773 - val_loss: 0.0815
Epoch 4/5
1688/1688 ————— 21s 8ms/step - accuracy: 0.9808 - loss: 0.0610 - val_accuracy: 0.9762 - val_loss: 0.0788
Epoch 5/5
1688/1688 ————— 9s 5ms/step - accuracy: 0.9849 - loss: 0.0484 - val_accuracy: 0.9788 - val_loss: 0.0748

```
1 # Exercise 6 Tensorflow - Step 5: Evaluate Model
2
3 test_loss, test_acc = model.evaluate(
4     x_test,
5     y_test
6 )
7
8 print("Test Accuracy:", test_acc)
```

313/313 1s 2ms/step - accuracy: 0.9763 - loss: 0.0779
Test Accuracy: 0.9763000011444092

```
1 # Exercise 6 Tensorflow - Step 6: Predict One Image
2
3 predictions = model.predict(x_test)
4
5 print("Predicted Digit:", predictions[0].argmax())
6 print("Actual Digit:", y_test[0])
```

313/313 1s 2ms/step
Predicted Digit: 7
Actual Digit: 7

```
1 # Exercise 6 PyTorch - Step 1: Import Libraries
2
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6
7 from torchvision import datasets, transforms
8 from torch.utils.data import DataLoader
```

```
1 # Exercise 6 PyTorch - Step 2: Load and Normalize MNIST
2
3 transform = transforms.Compose([
4     transforms.ToTensor(),
5     transforms.Normalize((0.1307,), (0.3081,))
6 ])
7
8 train_dataset = datasets.MNIST(
9     root='./data',
10    train=True,
11    download=True,
12    transform=transform
13 )
14
15 test_dataset = datasets.MNIST(
16     root='./data',
17     train=False,
18     download=True,
19     transform=transform
20 )
21
22 train_loader = DataLoader(
23     train_dataset,
24     batch_size=64,
25     shuffle=True
26 )
27
28 test_loader = DataLoader(
29     test_dataset,
30     batch_size=64,
31     shuffle=False
32 )
33
34 print("Training Samples:", len(train_dataset))
35 print("Testing Samples:", len(test_dataset))
```

100%|██████████| 9.91M/9.91M [00:00<00:00, 53.7MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 2.20MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 14.8MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 9.61MB/s]Training Samples: 60000
Testing Samples: 10000

```
1 # Exercise 6 PyTorch - Step 3: Create Neural Network
2
3 class MNISTNet(nn.Module):
4
5     def __init__(self):
6         super().__init__()
7
8         self.fc1 = nn.Linear(784,128)
9         self.relu = nn.ReLU()
```



```
10     self.fc2 = nn.Linear(128,10)
11
12     def forward(self,x):
13         x = x.view(-1,784)
14
15         x = self.fc1(x)
16         x = self.relu(x)
17         x = self.fc2(x)
18
19         return x
20
21 model = MNISTNet()
22
23 print(model)
```

```
MNISTNet(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)
```

```
1 # Exercise 6 PyTorch - Step 4: Loss and Optimizer
2
3 criterion = nn.CrossEntropyLoss()
4
5 optimizer = optim.Adam(
6     model.parameters(),
7     lr=0.001
8 )
```

```
1 # Exercise 6 PyTorch - Step 5: Train Model
2
3 for epoch in range(5):
4
5     running_loss = 0
6
7     for images, labels in train_loader:
8
9         optimizer.zero_grad()
10
11         outputs = model(images)
12
13         loss = criterion(outputs, labels)
14
15         loss.backward()
16
17         optimizer.step()
18
19         running_loss += loss.item()
20
21     print(f"Epoch {epoch+1}, Loss: {running_loss/len(train_loader):.4f}")
```

```
Epoch 1, Loss: 0.1090
Epoch 2, Loss: 0.0765
Epoch 3, Loss: 0.0570
Epoch 4, Loss: 0.0462
Epoch 5, Loss: 0.0367
```

```
1 # Exercise 6 PyTorch - Step 6: Evaluate Accuracy
2
3 correct = 0
4 total = 0
5
6 with torch.no_grad():
7
8     for images, labels in test_loader:
9
10         outputs = model(images)
11
12         _, predicted = torch.max(outputs.data,1)
13
14         total += labels.size(0)
15
16         correct += (predicted == labels).sum().item()
17
18 accuracy = 100 * correct / total
19
20 print(f"Test Accuracy: {accuracy:.2f}%")
```

Test Accuracy: 97.47%

```
1 # Exercise 6 PyTorch - Step 7: Predict Single Image
2
3 image, label = test_dataset[0]
```



```
4
5 with torch.no_grad():
6
7     output = model(image.unsqueeze(0))
8
9     prediction = torch.argmax(output, dim=1)
10
11 print("Predicted:", prediction.item())
12
13 print("Actual:", label)
```

Predicted: 7
Actual: 7

```
1 # Exercise 7 TensorFlow - Save Model
2
3 model.save("mnist_model.h5")
4 print("Model Saved Successfully")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file Model Saved Successfully

```
1 # Exercise 7 TensorFlow - Load Model
2
3 loaded_model = tf.keras.models.load_model("mnist_model.h5")
4
5 print("Model Loaded Successfully")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty Model Loaded Successfully

```
1 # Exercise 7 TensorFlow - Predict Using Loaded Model
2
3 predictions = loaded_model.predict(x_test)
4
5 print("Predicted Digit:", predictions[0].argmax())
6
7 print("Actual Digit:", y_test[0])
```

313/313 1s 2ms/step
Predicted Digit: 7
Actual Digit: 7

```
1 # Exercise 7 PyTorch - Save Model
2
3 torch.save(model.state_dict(), "mnist_model.pth")
4 print("Model Saved Successfully")
```

Model Saved Successfully

```
1 # Exercise 7 PyTorch - Load Model
2
3 loaded_model = MNISTNet()
4
5 loaded_model.load_state_dict(torch.load("mnist_model.pth"))
6
7 loaded_model.eval()
8
9 print("Model Loaded Successfully")
```

Model Loaded Successfully

```
1 # Exercise 7 PyTorch - Predict Using Loaded Model
2
3 image, label = test_dataset[0]
4
5 with torch.no_grad():
6
7     output = loaded_model(image.unsqueeze(0))
8
9     prediction = torch.argmax(output, dim=1)
10
11 print("Predicted:", prediction.item())
12
13 print("Actual:", label)
```

Predicted: 7
Actual: 7

```
1 # Exercise 8: Transfer Learning
2 # TensorFlow - MobileNetV2
3
```



```
4 import tensorflow as tf
5
6 base_model = tf.keras.applications.MobileNetV2(
7     weights='imagenet',
8     include_top=False,
9     input_shape=(224,224,3)
10 )
11
12 base_model.trainable = False
13
14 model = tf.keras.Sequential([
15     base_model,
16     tf.keras.layers.GlobalAveragePooling2D(),
17     tf.keras.layers.Dense(3, activation='softmax')
18 ])
19
20 model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_c9406464/9406464 0s 0us/step

Model: "sequential"

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense (Dense)	(None, 3)	3,843

Total params: 2,261,827 (8.63 MB)
Trainable params: 3,843 (15.01 KB)
Non-trainable params: 2,257,984 (8.61 MB)

```
1 # Verify Frozen Layers
2 print("Base Model Trainable:", base_model.trainable)
```

Base Model Trainable: False

```
1 # Exercise 8: Transfer Learning
2 # PyTorch - ResNet50
3
4 import torch
5 import torch.nn as nn
6 from torchvision import models
7
8 model = models.resnet50(
9     weights=models.ResNet50_Weights.DEFAULT
10 )
11
12 for param in model.parameters():
13     param.requires_grad = False
14
15 num_features = model.fc.in_features
16
17 model.fc = nn.Linear(num_features, 3)
18
19 print(model)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
```

```
)
(2): Bottleneck(
  (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
)
)
(layer2): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

```
1 # Verify Final Layer
2 print(model.fc)
```

```
Linear(in_features=2048, out_features=3, bias=True)
```

```
1 # Verify Trainable Parameters
2
3 trainable = sum(
4     p.numel()
5     for p in model.parameters()
6     if p.requires_grad
7 )
8
9 print(
10     "Trainable Parameters:",
11     trainable
12 )
```

```
Trainable Parameters: 6147
```

```
1 # Exercise 9: Custom Training Loop
2 # TensorFlow - Step 1: Generate Random Data
3
4 import tensorflow as tf
5 import numpy as np
6
7 x_train = np.random.rand(1000, 1).astype(np.float32) * 10
8
9 y_train = 2 * x_train + 5
```

```
1 # TensorFlow - Step 2: Create Linear Regression Model
2 model = tf.keras.Sequential([
3     tf.keras.layers.Dense(1)
4 ])
```

```
1 # TensorFlow - Step 3: Define Loss and Optimizer
2 loss_fn = tf.keras.losses.MeanSquaredError()
3
4 optimizer = tf.keras.optimizers.SGD(
5     learning_rate=0.01
6 )
```

```
1 # TensorFlow - Step 4: Custom Training Loop
2 epochs = 1000
3
4 for epoch in range(epochs):
5
6     with tf.GradientTape() as tape:
7         predictions = model(x_train)
8         loss = loss_fn(y_train, predictions)
9
10    gradients = tape.gradient(loss, model.trainable_variables)
11    optimizer.apply_gradients(zip(gradients, model.trainable_variables))
12
```

```
13     if (epoch + 1)% 100 == 0:
14         print(f"Epoch {epoch+1}, Loss: {loss.numpy():.6f}")

Epoch 100, Loss: 1.883075
Epoch 200, Loss: 0.682636
Epoch 300, Loss: 0.247464
Epoch 400, Loss: 0.089709
Epoch 500, Loss: 0.032521
Epoch 600, Loss: 0.011789
Epoch 700, Loss: 0.004274
Epoch 800, Loss: 0.001549
Epoch 900, Loss: 0.000562
Epoch 1000, Loss: 0.000204
```

```
1 # TensorFlow - Step 5: Print Learned Weight and Bias
2 weight = model.layers[0].kernel.numpy()[0][0]
3 bias = model.layers[0].bias.numpy()[0]
4
5 print("Learned Weight:", weight)
6 print("Learned Bias:", bias)

Learned Weight: 2.004202
Learned Bias: 4.972096
```

```
1 # Exercise 9: Custom Training Loop
2 # PyTorch - Step 1: Generate Data
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6
7 x_train = torch.rand(1000,1) * 10
8
9 y_train = 2 * x_train + 5
```

```
1 # PyTorch - Step 2: Linear Regression Model
2 model = nn.Linear(
3     1,
4     1
5 )
```

```
1 # PyTorch - Step 3: Loss and Optimizer
2 criterion = nn.MSELoss()
3
4 optimizer = optim.SGD(
5     model.parameters(),
6     lr=0.01
7 )
```

```
1 # PyTorch - Step 4: Custom Training Loop
2 epochs = 1000
3
4 for epoch in range(epochs):
5
6     predictions = model(x_train)
7     loss = criterion(predictions, y_train)
8
9     optimizer.zero_grad()
10    loss.backward()
11    optimizer.step()
12
13    if (epoch + 1) % 100 == 0:
14        print(f"Epoch {epoch+1}, Loss: {loss.item():.6f}")

Epoch 100, Loss: 1.376604
Epoch 200, Loss: 0.512567
Epoch 300, Loss: 0.190850
Epoch 400, Loss: 0.071061
Epoch 500, Loss: 0.026459
Epoch 600, Loss: 0.009852
Epoch 700, Loss: 0.003668
Epoch 800, Loss: 0.001366
Epoch 900, Loss: 0.000509
Epoch 1000, Loss: 0.000189
```

```
1 # PyTorch - Step 5: Print Learned Weight and Bias
2 weight = model.weight.item()
3 bias = model.bias.item()
4
5 print("Learned Weight:", weight)
6 print("Learned Bias:", bias)

Learned Weight: 2.004115104675293
Learned Bias: 4.972721099853516
```

